

Heaven's light is our guide

Rajshahi University of Engineering & Technology (RUET)

Rajshahi-6204, Bangladesh

Department of Computer Science & Engineering

CSE 3206: Software Engineering Sessional

Lab 3: Design Pattern Analysis, Implementation and Code Review

DELIVERABLE 5

Code Review Report

Iterator & Mediator — CampusConnect (C++17)

Presentation Group 7 — Assigned Patterns: Iterator & Mediator

Name	Student ID	GitHub Username
Abdullah Al Kafi (Group Leader)	2203139	@Kafi2611
Sadaf Rahman	2203140	@sadaf532
Sakila Akter	2203141	@SakilaAkter

GitHub Repository: <https://github.com/Kafi2611/CSE3206-Group7-Iterator-Mediator>

Submitted to: Course Teacher, CSE 3206 — Department of CSE, RUET

Contents

1. Review Scope and Method.....	3
1.1 What was reviewed.....	3
1.2 Review process.....	3
2. Automated Check Results.....	4
3. Checklist Review (10 criteria).....	5
3.1 Score Summary.....	6
3.2 Complexity of the Main Operations.....	7
4. Findings and Fixes Log.....	7
5. Unit Test Report.....	8
6. Recommendations (Future Work).....	9
7. Peer Review Feedback (to be completed after the presentation).....	9

1. Review Scope and Method

1.1 What was reviewed

File	Contents	Lines
iterator/include/Iterator.hpp	Iterator interface + exception	56
iterator/include/SocialNetwork.hpp	Aggregate interface	46
iterator/include/CampusConnect.hpp	Concrete aggregate (declaration)	61
iterator/src/CampusConnect.cpp	Concrete aggregate	101
iterator/include/FeedIterators.hpp	FailFastIterator + concrete iterators	114
iterator/src/FeedIterators.cpp	Traversal algorithms	97
iterator/include/FeedScreen.hpp	Client (declaration)	42
iterator/src/FeedScreen.cpp	Client	60
iterator/main.cpp	Iterator demo	117
mediator/include/Message.hpp	Message, DeliveryStatus, PermissionDenied	51
mediator/include/ChatMediator.hpp	Mediator interface	48
mediator/include/User.hpp	Colleague (declaration)	63
mediator/src/User.cpp	Colleague	65
mediator/src/Member.cpp	Concrete colleague	24
mediator/src/SupportBot.cpp	Concrete colleague (bot)	30
mediator/include/MessengerServer.hpp	Concrete mediator (declaration)	68
mediator/src/MessengerServer.cpp	Concrete mediator	187
mediator/main.cpp	Mediator demo	103
tests/test_iterator.cpp	16 unit tests	203
tests/test_mediator.cpp	17 unit tests	202

Small headers not listed (`Models.hpp`, `Member.hpp`, `SupportBot.hpp`, `TestFramework.hpp`) were reviewed as well. Build scripts, CMake files and the CI workflow were checked for correctness on Linux and for Windows compatibility.

1.2 Review process

The review combined **manual cross-review** (each module is read by a member who did not write it) with **automated tools**, and was carried out against the 10-point Code Review Checklist from the lab handout.

Module	Main author	Reviewer
Iterator – CampusConnect Feed	Abdullah Al Kafi	Sadaf Rahman
Mediator – CampusConnect Messenger	Sadaf Rahman	Abdullah Al Kafi
Unit tests, build scripts, documentation	Sakila Akter	Abdullah Al Kafi and Sadaf Rahman

Rating scale used in Section 3: 5 = Excellent, 4 = Good, 3 = Satisfactory, 2 = Needs improvement, 1 = Poor.

2. Automated Check Results

Check	Command / tool	Result
Build (GCC 13)	<code>g++ -std=c++17 -Wall -Wextra -Wpedantic -Wshadow -Wconversion</code>	0 warnings
Build (Clang 18)	same flags plus <code>-Wnon-virtual-dtor -Wold-style-cast -Woverloaded-virtual</code>	0 warnings
Unit tests	<code>test_iterator, test_mediator</code> (via <code>ctest</code>)	33 / 33 passed
Line coverage	<code>g++ --coverage + gcov</code> on module <code>.cpp</code> files	Iterator 100 % , Mediator 98.9 % (2 compiler-generated lines)
Static analysis	clang-tidy 18: bugprone, modernize, performance, readability, cppcoreguidelines, misc	13 warnings → 6 fixed, 7 accepted (see F3–F5, F9)
Memory errors / undefined behaviour	<code>-fsanitize=address, undefined</code> on tests and demos	0 errors
Memory leaks	<code>valgrind --leak-check=full</code> on demos and tests	0 leaks , 0 errors
Continuous integration	GitHub Actions: CMake configure → build → ctest → demos	runs on every push and pull request

Commands

```
# how to reproduce the main checks (Linux / Git Bash)
./build.sh                                # build + 33 unit tests
g++ -std=c++17 -fsanitize=address,undefined -Iiterator/include -Itests \
    iterator/src/*.cpp tests/test_iterator.cpp -o t && ./t
clang-tidy iterator/src/*.cpp mediator/src/*.cpp \
    -checks='-*,bugprone-*,modernize-*,performance-*,readability-*,cppcoreguidelines-*,misc-*' \
    -- -std=c++17 -Iiterator/include -Imediator/include
```

3. Checklist Review (10 criteria)

1. Naming Convention — 5 / 5

- Classes in PascalCase (`FriendsIterator`, `MessengerServer`), functions in camelCase (`createNewsFeedIterator`, `presenceChanged`), private members end with `_` (`users_`, `cursor_`).
- Names reveal intent: `ensureNotModified()`, `pendingCount()`, `isAutomated()`; booleans read as questions (`isSender`, `isOnServer`).
- Namespaces `campus::feed` and `campus::chat` separate the modules; file names match the main class they contain.

2. SOLID Principles — 4 / 5

- **S** – storage (`CampusConnect`), traversal (iterators) and display (`FeedScreen`) are separate; users only represent people, the server owns the rules.
- **O** – a new iterator or a new colleague type (e.g. `SupportBot`) was added without modifying existing classes.
- **L** – any `Iterator<T>` works in `FeedScreen`; `Member` and `SupportBot` are used wherever a `User&` is expected.
- **I** – small, focused interfaces (`Iterator<T>`: 3 methods).
- **D** – clients depend on `SocialNetwork / Iterator<T>`; users depend on `ChatMediator`, not on `MessengerServer`.

Improvement: `MessengerServer` has several duties (routing, groups, blocks, queue). Acceptable for a mediator, but it should be split (e.g. `GroupManager`, `DeliveryPolicy`) if more rules are added.

3. Readability — 5 / 5

- Consistent 4-space formatting; every file starts with a Doxygen header naming its pattern role (`<<Iterator>>`, `<<ConcreteMediator>>`).
- Short functions – the longest (`sendToGroup`) is about 35 lines and is split into validation, recording and delivery.
- Sample data and demo steps are labelled so the live demo follows the documentation.

4. Object Interaction — 5 / 5

- Iterator: client → aggregate (factory) → iterator → aggregate (read-only); the client never touches containers.
- Mediator: colleagues call only `ChatMediator`; the server calls `receive()` on colleagues. No `User` holds a pointer or reference to another `User` (verified by reading all colleague code).
- Sequence diagrams in the documentation match the real call order in the code.

5. Pattern Correctness — 5 / 5

- All GoF participants are present for both patterns (see the mapping tables in the documentation).
- Iterator: external, polymorphic iterators; representation hidden; several independent cursors (tested).
- Mediator: colleagues are decoupled; interaction rules live only in the concrete mediator; new colleague types plug in (bot).

6. Reusability — 4 / 5

- Templates `Iterator<T>` and `FailFastIterator<T, Key>` serve both profiles and posts.
- Output is injected (`std::ostream&`) so the same classes work in the console, in tests or in another UI.
- Each module is an independent CMake library (`campus_feed`, `campus_chat`).

Improvement: Our `Iterator<T>` is not compatible with range-based `for` / `<algorithm>`; a small `begin()/end()` adapter would make it reusable with the STL.

7. Exception Handling — 5 / 5

- Invalid input is rejected with standard exceptions and clear messages (`invalid_argument`, `out_of_range`, `logic_error`).
- Domain-specific exceptions: `ConcurrentModificationError`, `PermissionDenied`.
- `User::join()` gives the strong guarantee (pointer stored only after successful registration); `leave()` and `unregisterUser()` are `noexcept`; validation happens before any state change.
- Every error path is covered by at least one unit test (`CHECK_THROWS_AS`).

8. Documentation — 5 / 5

- Doxygen comments on all public classes and methods, including thrown exceptions.
- README with quick start, lab documentation (15-section format), UML class, sequence and motivation diagrams, and a GitHub guide.

9. Maintainability — 4 / 5

- Modular folders, one-class-per-file for main classes, CMake + one-click scripts, CI on every push.
- 33 unit tests and zero warnings make refactoring safe.

Improvement: The server's raw non-owning `User*` registry relies on a lifetime rule (server outlives users); documented and enforced by RAII `leave()`, but `std::weak_ptr` would make it impossible to misuse.

10. Efficiency — 4 / 5

- Iterators store keys/indices, not copies of profiles or posts; `next()` is $O(\log n)$ (map lookup) or $O(1)$ (vector index).
- `std::map/std::set` give $O(\log n)$ lookups for users, friends, groups and block lists.
- No unnecessary copies: `const&` parameters, `std::move` for strings and queues.

Improvement: Iterators build their order eagerly: `NewsFeedIterator` scans all posts ($O(P + m \log m)$) on creation. For a very large network a per-author post index and lazy/paged iteration would be better.

3.1 Score Summary

Criterion	Score
1. Naming Convention	5 / 5
2. SOLID Principles	4 / 5
3. Readability	5 / 5
4. Object Interaction	5 / 5
5. Pattern Correctness	5 / 5
6. Reusability	4 / 5
7. Exception Handling	5 / 5
8. Documentation	5 / 5
9. Maintainability	4 / 5
10. Efficiency	4 / 5
Total	46 / 50 (92 %)

3.2 Complexity of the Main Operations

Operation	Time	Notes
<code>createFriendsIterator(id)</code>	$O(f)$	f = number of friends (copy of ids from a <code>std::set</code>)
<code>createSuggestionsIterator(id)</code>	$O(\sum \text{deg} + k \log k)$	BFS depth 2 over friends' friends; k suggestions sorted
<code>createNewsFeedIterator(id)</code>	$O(P + m \log m)$	P = all posts scanned, m = posts kept and sorted
<code>next()</code> / <code>hasNext()</code>	$O(\log n)$ / $O(1)$	profile lookup in <code>std::map</code> ; post lookup by index
<code>sendDirect()</code>	$O(\log n)$	map lookups for user and block list
<code>sendToGroup()</code>	$O(g \log n)$	g = group size
<code>presenceChanged()</code> (flush)	$O(q)$	q = queued messages

4. Findings and Fixes Log

Issues found during review and how they were resolved. Severity: **High** = wrong behaviour, **Medium** = design or robustness risk, **Low** = style or guideline, **Info** = accepted with a reason.

ID	Sev.	Location	Finding	Action / status
F1	Medium	FeedIterators.hpp	<code>FailFastIterator</code> exposed <code>network_</code> as a protected data member (subclasses could bypass the fail-fast check).	Made private; added a protected read-only accessor <code>network()</code> . Fixed
F2	Medium	MessengerServer.cpp <code>sendDirect</code>	The returned status re-read <code>recipient.isOnline()</code> after delivery; a <code>receive()</code> callback could change presence and misreport the result.	<code>deliver()</code> now returns the status of the action it actually took. Fixed
F3	Low	Iterator.hpp, SocialNetwork.hpp, ChatMediator.hpp	clang-tidy <code>cppcoreguidelines-special-member-functions</code> : polymorphic bases without declared copy/move (slicing risk).	Protected defaulted copy/move operations (Core Guidelines C.67). Fixed
F4	Low	User.hpp	Copy was deleted but move was not declared, although the server stores the user's address.	Move constructor/assignment explicitly deleted. Fixed
F5	Low	Message.hpp	clang-tidy <code>performance-enum-size</code> on <code>DeliveryStatus</code> .	Underlying type <code>std::uint8_t</code> . Fixed
F6	Low	SocialNetwork.hpp, CampusConnect.hpp	A caller could create an iterator and accidentally ignore it.	Factory methods marked <code>[[nodiscard]]</code> . Fixed
F7	Info	Member.hpp/.cpp	Redundant destructor calling <code>leave()</code> (the base destructor already does it).	Removed. Fixed
F8	Low	build.bat	Saved with LF line endings; Windows <code>cmd</code> can mis-handle labels/goto in LF batch files.	Converted to CRLF and added <code>.gitattributes</code> so Git keeps it CRLF. Fixed
F9	Info	Message.hpp	clang-tidy <code>misc-non-private-member-variables-in-classes</code> (7 warnings) on <code>Message</code> fields.	Accepted: <code>Message</code> is a plain data record like <code>Profile/Post</code> ; getters would add noise without protecting any invariant.

ID	Sev.	Location	Finding	Action / status
F10	Info	MessengerServer.hpp	Registry stores non-owning <code>User*</code> ; a user destroyed without leaving would dangle.	Mitigated: <code>User::~User()</code> calls <code>leave()</code> ; server declared first in <code>main</code> ; covered by test <code>destroyed_user_unregisters_automatically</code> .
F11	Low	tests	Coverage showed untested branches (empty-collection output, empty group message/id, empty user id, user registered on another server, going online with an empty queue).	4 tests added (29 → 33); coverage raised to 100 % / 98.9 %. Fixed

5. Unit Test Report

All **33** tests pass (Iterator 16, Mediator 17). Test names describe the expected behaviour, so the list doubles as a specification.

#	Iterator test (test_iterator.cpp)	Result
1	<code>friends_iterator_returns_direct_friends_in_id_order</code>	PASS
2	<code>friends_iterator_on_profile_without_friends_is_empty</code>	PASS
3	<code>friendship_is_mutual</code>	PASS
4	<code>suggestions_rank_by_mutual_friends_and_exclude_existing_friends</code>	PASS
5	<code>suggestions_never_include_self</code>	PASS
6	<code>news_feed_is_newest_first_and_only_from_friend_circle</code>	PASS
7	<code>publish_post_validates_input</code>	PASS
8	<code>reset_allows_a_second_traversal</code>	PASS
9	<code>next_after_end_throws_out_of_range</code>	PASS
10	<code>two_iterators_on_same_collection_are_independent</code>	PASS
11	<code>modifying_network_makes_live_iterator_fail_fast</code>	PASS
12	<code>unknown_profile_is_rejected</code>	PASS
13	<code>network_rejects_bad_profiles_and_friendships</code>	PASS
14	<code>client_uses_the_same_loop_for_every_iterator</code>	PASS
15	<code>client_prints_placeholder_for_empty_collections</code>	PASS
16	<code>mutual_friends_uses_two_iterators</code>	PASS

#	Mediator test (test_mediator.cpp)	Result
1	<code>direct_message_to_online_user_is_delivered</code>	PASS
2	<code>message_to_offline_user_is_queued_then_flushed_in_order</code>	PASS
3	<code>blocked_sender_cannot_reach_recipient</code>	PASS
4	<code>invalid_direct_messages_are_rejected</code>	PASS

#	Mediator test (test_mediator.cpp)	Result
5	going_online_with_nothing_queued_is_harmless	PASS
6	group_message_reaches_all_members_except_sender	PASS
7	group_message_respects_block_list_and_offline_queue	PASS
8	non_member_cannot_post_in_group	PASS
9	create_group_validates_members	PASS
10	group_and_user_input_is_validated	PASS
11	support_bot_replies_through_the_mediator	PASS
12	support_bot_ignores_group_messages_and_other_bots	PASS
13	user_must_join_before_chatting	PASS
14	duplicate_ids_and_double_join_are_rejected	PASS
15	destroyed_user_unregisters_automatically	PASS
16	server_rejects_users_registered_elsewhere	PASS
17	history_records_every_accepted_message_once	PASS

6. Recommendations (Future Work)

- Add a `begin()/end()` adapter so `Iterator<T>` works with range-based `for` and `<algorithm>`.
- Make `NewsFeedIterator` lazy/paged and keep a per-author post index for large networks.
- If more messenger rules are added, split `MessengerServer` into smaller collaborators (e.g. `GroupManager`, `DeliveryPolicy` via the Strategy pattern) to avoid a God object.
- Replace the raw `User*` registry with `std::weak_ptr` if users become shared objects.
- For a real server: thread safety, asynchronous delivery and persistent message history.
- Optionally migrate the tests to GoogleTest or Catch2 and publish coverage in CI.

7. Peer Review Feedback (to be completed after the presentation)

Audience groups review our implementation with the same checklist. Record their questions and suggestions here, together with our response, then export this report again as PDF.

Reviewer group	Checklist item	Comment / question	Our response / action

Reviewed and approved by Group 7:

Name	Student ID	Role in review	Signature
Abdullah Al Kafi	2203139	Reviewer of the Mediator module	
Sadaf Rahman	2203140	Reviewer of the Iterator module	
Sakila Akter	2203141	Reviewer of tests, scripts and documents	